

Module 06 — Pourquoi Excel ne suffit plus : introduction à SQL

Bootcamp Data Analyst — From Zero to Hero | Niveau Débutant · Acte III

Ce que tu seras capable de faire à la fin de ce module

- Expliquer pourquoi SQL est indispensable quand Excel atteint ses limites
- Comprendre ce qu'est une base de données relationnelle
- Utiliser SQLiteOnline sans rien installer
- Écrire des requêtes `SELECT`, `WHERE`, `ORDER BY`, `LIMIT`
- Filtrer des données avec `AND`, `OR`, `IN`, `BETWEEN`, `LIKE`

 **Durée estimée** : 60 minutes

 **Outil** : [SQLiteOnline](#) — 100% navigateur, zéro installation

 **Prérequis** : Acte II complété — tu maîtrises Excel et les formules de base

1. Le problème — les limites concrètes d'Excel

Tu sors du Mini-projet A avec une belle maîtrise d'Excel. Mais imagine maintenant :

Ton opérateur télécom a **8 millions d'abonnés**, **500 millions de transactions** par an, réparties sur **12 tables différentes** mises à jour en temps réel par 200 collègues simultanément. Ton manager te demande : "Combien de clients Abidjan ont rechargé plus de 3 fois ce mois-ci ?"

Excel répond comment à ça ?

LIMITE	EXCEL	SQL
Volume de données	~1 million de lignes max, ralentit à partir de 100k	Des milliards de lignes
Données en temps réel	Fichier statique	Connecté à la base de données live
Plusieurs tables	RECHERCHEV fastidieux, fragile	JOIN natif en une ligne
Collaboration	Un utilisateur à la fois (sans Sheets)	Des centaines simultanément
Reproductibilité	Clics manuels, non traçables	Code écrit, auditable, réutilisable
Automatisation	Macros complexes	Une requête planifiée

Excel et Google Sheets restent indispensables pour la **communication et la visualisation**. Mais pour **interroger des données à grande échelle**, il faut SQL.

💡 En entretien DA, la question "Tu connais SQL ?" arrive dans 95% des offres. Ce n'est pas un bonus — c'est un prérequis.

2. Qu'est-ce qu'une base de données relationnelle ?

Définition

Une **base de données relationnelle** est un ensemble de **tables** organisées et reliées entre elles. Chaque table ressemble à un tableau Excel — des lignes (enregistrements) et des

colonnes (champs) — mais avec des règles strictes qui garantissent la cohérence des données.

L'analogie du classeur Excel

Excel / Google Sheets		Base de données relationnelle
Classeur (fichier .xlsx)	→	Base de données
Feuille (onglet)	→	Table
Ligne	→	Enregistrement (row)
Colonne	→	Champ (field / column)
RECHERCHEV entre feuilles	→	JOIN entre tables
Formule	→	Requête SQL

Un exemple concret — base télécom

```
Base de données : ansut_telecom
|
|— Table clients      (200 lignes)  → qui sont nos clients ?
|   id_client | nom | région | segment
|
|— Table transactions (500 lignes)  → que font-ils ?
|   id_transaction | id_client | date | montant | produit
|
|— Table forfaits     (10 lignes)   → quels produits proposons-nous ?
|   id_forfait | nom | prix | data_go
```

La colonne `id_client` dans `transactions` pointe vers `id_client` dans `clients`. C'est ce qu'on appelle une **clé étrangère** — le lien qui relie les tables entre elles. On y reviendra en M08.

Les SGBD les plus courants

SGBD	USAGE PRINCIPAL
SQLite	Embarqué, léger, parfait pour apprendre
PostgreSQL	Open source, très répandu en entreprise
MySQL / MariaDB	Web, startups, WordPress
Microsoft SQL Server	Grandes entreprises, écosystème Microsoft
BigQuery	Google Cloud, données massives
Snowflake	Data warehouse cloud

💡 La syntaxe SQL est à 90% identique entre tous ces SGBD. Ce que tu apprends ici sur SQLite fonctionne sur PostgreSQL, MySQL et BigQuery avec des ajustements mineurs.

3. Qu'est-ce que SQL ?

SQL = Structured Query Language — Langage de Requête Structuré.

C'est le langage qui permet de **communiquer avec une base de données** : lire, filtrer, agréger, modifier et supprimer des données.

SQL en une phrase

SQL, c'est la façon de dire à une base de données : "Donne-moi exactement ces données, organisées comme ça, avec ces conditions."

Les 4 opérations fondamentales (CRUD)

OPÉRATION	COMMANDE SQL	RÔLE
Create	<code>INSERT INTO</code>	Ajouter des données
Read	<code>SELECT</code>	Lire / interroger les données
Update	<code>UPDATE</code>	Modifier des données
Delete	<code>DELETE</code>	Supprimer des données

En tant que **Data Analyst**, tu utiliseras quasi exclusivement `SELECT` — lire et analyser des données sans jamais les modifier. C'est ce sur quoi on se concentre dans tout l'Acte III.

À quoi ressemble du SQL ?

SQL

```
-- Combien de clients Abidjan ont rechargé plus de 3 fois ce mois ?  
SELECT  
    c.nom,  
    COUNT(t.id_transaction) AS nb_recharges  
FROM clients c  
JOIN transactions t ON c.id_client = t.id_client  
WHERE c.région = 'Abidjan'  
    AND t.date >= '2025-03-01'  
GROUP BY c.nom  
HAVING COUNT(t.id_transaction) > 3  
ORDER BY nb_recharges DESC;
```

Cette requête semble complexe aujourd'hui — tu la comprendras et écriras seul à la fin de l'Acte III.

💡 SQL se lit presque comme de l'anglais : "Sélectionne le nom et le nombre de transactions, depuis la table clients jointe à transactions, où la région est Abidjan et la date est en mars, regroupé par nom, en gardant ceux avec plus de 3 transactions, trié par ordre décroissant."

4. L'environnement — SQLiteOnline

On utilise **SQLiteOnline** — 100% navigateur, zéro installation.

Accès

→ Ouvrir : <https://sqliteonline.com>

Interface

[Panneau gauche]	[Éditeur SQL]	[Résultats]
Tables de la base	SELECT * FROM	id nom ..
> clients	clients;	1 Koné
> transactions		2 Traoré
> forfaits		

Importer le dataset du Mini-projet B

1. Aller sur <https://sqliteonline.com>
2. Cliquer sur "Import" (menu en haut)
3. Importer les 3 fichiers CSV un par un :
 - clients.csv
 - transactions.csv
 - forfaits.csv
4. Les tables apparaissent dans le panneau gauche
5. Tu peux maintenant écrire des requêtes dans l'éditeur

ACTION	RACCOURCI
Exécuter la requête	Ctrl + Entrée ou bouton ► Run
Nouvelle ligne	Entrée
Commenter une ligne	-- en début de ligne

💡 Pour ce module et les suivants, ouvre SQLiteOnline dans un onglet et pratique chaque requête en temps réel. SQL s'apprend en écrivant, pas en lisant.

5. **SELECT** / **FROM** — lire des données

C'est la base de toute requête SQL. Tout commence par **SELECT**.

Syntaxe

```
SQL
SELECT colonne1, colonne2, ...
FROM nom_table;
```

Exemples sur notre dataset télécom

```
SQL
-- Voir tous les clients (toutes les colonnes)
SELECT *
FROM clients;

-- Voir seulement le nom et la région
SELECT nom, région
FROM clients;

-- Renommer une colonne dans le résultat avec AS
SELECT nom AS client_name, région AS zone
FROM clients;

-- Voir toutes les transactions
SELECT *
FROM transactions;
```

Conventions d'écriture SQL

SQL n'est pas sensible à la casse — `select` = `SELECT`. Mais par convention :

```
SQL
-- ✅ Convention recommandée
SELECT nom, région      -- mots-clés SQL en MAJUSCULES
FROM clients            -- noms de tables et colonnes en minuscules
WHERE région = 'Abidjan'; -- chaînes de texte entre guillemets simples
                        -- toujours terminer par ;

-- Commentaire sur une ligne
/* Commentaire
   sur plusieurs lignes */
```

`SELECT *` — puissant mais à utiliser avec précaution

`SELECT *` retourne **toutes les colonnes**. Utile pour explorer, mais en production on préfère lister explicitement les colonnes dont on a besoin — plus lisible, plus performant sur de grandes tables.



SELECT * c'est comme demander "tout le menu" au restaurant. En exploration c'est bien, en production on commande à la carte.

6. **WHERE** — filtrer les données

WHERE est l'équivalent SQL du filtre Excel ou du **SI()**. Il ne retourne que les lignes qui satisfont une condition.

Syntaxe

SQL

```
SELECT colonne1, colonne2
FROM nom_table
WHERE condition;
```

Exemples

```
SQL
-- Clients d'Abidjan seulement
SELECT *
FROM clients
WHERE région = 'Abidjan';

-- Transactions supérieures à 1 000 FCFA
SELECT *
FROM transactions
WHERE montant > 1000;

-- Transactions échouées
SELECT *
FROM transactions
WHERE statut = 'Échoué';

-- Clients qui NE sont PAS d'Abidjan
SELECT *
FROM clients
WHERE région != 'Abidjan';
-- ou
WHERE région <> 'Abidjan';
```

Opérateurs de comparaison

OPÉRATEUR	SIGNIFICATION	EXEMPLE
<code>=</code>	Égal	<code>WHERE région = 'Abidjan'</code>
<code>!=</code> ou <code><></code>	Différent	<code>WHERE statut != 'Échoué'</code>
<code>></code>	Supérieur	<code>WHERE montant > 5000</code>
<code>>=</code>	Supérieur ou égal	<code>WHERE montant >= 5000</code>
<code><</code>	Inférieur	<code>WHERE montant < 1000</code>
<code><=</code>	Inférieur ou égal	<code>WHERE montant <= 1000</code>
<code>IS NULL</code>	Valeur manquante	<code>WHERE évaluation IS NULL</code>
<code>IS NOT NULL</code>	Valeur présente	<code>WHERE montant IS NOT NULL</code>

7. **AND**, **OR**, **IN**, **BETWEEN**, **LIKE** — affiner les filtres

AND — toutes les conditions doivent être vraies

```
SQL
-- Clients d'Abidjan ET segment Postpayé
SELECT *
FROM clients
WHERE région = 'Abidjan'
      AND segment = 'Postpayé';

-- Transactions réussies ET supérieures à 2 000 FCFA
SELECT *
FROM transactions
WHERE statut = 'Réussi'
      AND montant > 2000;
```

OR — au moins une condition doit être vraie

```
SQL
-- Clients d'Abidjan OU de Bouaké
SELECT *
FROM clients
WHERE région = 'Abidjan'
      OR région = 'Bouaké';
```

IN — valeur dans une liste (remplace plusieurs OR)

SQL

```
-- Équivalent à OR mais plus lisible
SELECT *
FROM clients
WHERE région IN ('Abidjan', 'Bouaké', 'San Pedro');

-- Transactions sur certains produits
SELECT *
FROM transactions
WHERE type_produit IN ('Forfait internet', 'Recharge 500');

-- Exclure des valeurs avec NOT IN
SELECT *
FROM transactions
WHERE statut NOT IN ('Échoué', 'En attente');
```

BETWEEN — valeur dans un intervalle

SQL

```
-- Transactions entre 1 000 et 5 000 FCFA (inclus)
SELECT *
FROM transactions
WHERE montant BETWEEN 1000 AND 5000;

-- Transactions de mars 2025
SELECT *
FROM transactions
WHERE date BETWEEN '2025-03-01' AND '2025-03-31';
```

LIKE — recherche par motif dans du texte

SQL

```
-- Clients dont le nom commence par 'Kone'
SELECT *
FROM clients
WHERE nom LIKE 'Kone%';    -- % = n'importe quoi après

-- Clients dont le nom contient 'ou'
SELECT *
FROM clients
WHERE nom LIKE '%ou%';    -- % de chaque côté = contient

-- Produits qui se terminent par 'internet'
SELECT *
FROM transactions
WHERE type_produit LIKE '%internet';

-- _ remplace exactement un caractère
WHERE code LIKE 'AB_123'; -- AB suivi d'un caractère puis 123
```

⚠ **LIKE** est sensible à la casse sur certains SGBD (PostgreSQL). Sur SQLite, il est insensible à la casse pour les caractères ASCII.

8. ORDER BY — trier les résultats

Syntaxe

SQL

```
SELECT colonne1, colonne2
FROM nom_table
WHERE condition
ORDER BY colonne [ASC | DESC];
```

Exemples

SQL

```
-- Transactions triées par montant décroissant
SELECT *
FROM transactions
ORDER BY montant DESC;

-- Clients triés par nom alphabétique
SELECT *
FROM clients
ORDER BY nom ASC; -- ASC est optionnel, c'est le défaut

-- Trier sur plusieurs colonnes
-- D'abord par région, puis par nom au sein de chaque région
SELECT *
FROM clients
ORDER BY région ASC, nom ASC;

-- Transactions récentes en premier
SELECT *
FROM transactions
ORDER BY date DESC;
```

MOT-CLÉ	SENS	DÉFAUT
ASC	Croissant (A→Z, 0→9, ancien→récent)	✓ Oui
DESC	Décroissant (Z→A, 9→0, récent→ancien)	✗ Non

9. **LIMIT** — limiter le nombre de résultats

Syntaxe

SQL

```
SELECT colonne1, colonne2
FROM nom_table
ORDER BY colonne DESC
LIMIT n;
```

Exemples

SQL

```
-- Les 5 premières lignes (exploration rapide)
SELECT *
FROM transactions
LIMIT 5;

-- Les 10 transactions les plus élevées
SELECT *
FROM transactions
ORDER BY montant DESC
LIMIT 10;

-- Le client inscrit le plus récemment
SELECT *
FROM clients
ORDER BY date_inscription DESC
LIMIT 1;
```

Différences selon le SGBD

SGBD	SYNTAXE
SQLite, MySQL, PostgreSQL	<code>LIMIT 10</code>
SQL Server	<code>SELECT TOP 10 ...</code>
Oracle	<code>WHERE ROWNUM <= 10</code>



LIMIT est le premier réflexe sur une nouvelle table inconnue : `SELECT * FROM table LIMIT 5;` pour voir à quoi ressemblent les données avant d'aller plus loin.

10. **DISTINCT** — supprimer les doublons

Syntaxe

```
SQL
SELECT DISTINCT colonne
FROM nom_table;
```

Exemples

```
SQL
-- Quelles régions existent dans la table clients ?
SELECT DISTINCT région
FROM clients;

-- Quels types de produits sont vendus ?
SELECT DISTINCT type_produit
FROM transactions;

-- Quelles combinaisons région + segment existent ?
SELECT DISTINCT région, segment
FROM clients
ORDER BY région;
```



SELECT DISTINCT est souvent le premier réflexe pour explorer les valeurs uniques d'une colonne catégorielle — équivalent de "Valeurs uniques" dans Excel ou `df['col'].unique()` en Python.

11. NULL — l'absence de valeur

NULL n'est pas zéro. Ce n'est pas une chaîne vide. C'est l'**absence totale de valeur** — "on ne sait pas". C'est l'un des concepts les plus mal compris en SQL.

Le piège classique

SQL

```
-- ❌ Ne retourne JAMAIS de résultat — même si des valeurs sont NULL
SELECT * FROM clients WHERE évaluation = NULL;
SELECT * FROM clients WHERE évaluation != NULL;

-- ✅ La seule façon correcte de tester NULL
SELECT * FROM clients WHERE évaluation IS NULL;
SELECT * FROM clients WHERE évaluation IS NOT NULL;
```

Pourquoi `= NULL` ne fonctionne pas ? Parce que NULL n'est pas une valeur — on ne peut pas la comparer. `NULL = NULL` retourne NULL, pas VRAI.

NULL vs zéro vs chaîne vide

VALEUR	SIGNIFICATION	EXEMPLE
NULL	Valeur inconnue / absente	Salaire non renseigné
0	Zéro — valeur numérique connue	Salaire = 0 FCFA (improbable mais possible)
''	Chaîne vide — valeur textuelle connue	Nom = "" (champ rempli mais vide)

NULL dans les calculs

```
SQL
-- Tout calcul avec NULL retourne NULL
SELECT montant + NULL      -- → NULL
SELECT montant * NULL      -- → NULL

-- COALESCE : remplacer NULL par une valeur par défaut
SELECT COALESCE(évaluation, 0) AS eval_avec_defaut
FROM clients;
-- → Si évaluation est NULL, retourne 0. Sinon retourne évaluation.
```

Sur notre dataset

```
SQL
-- Clients sans évaluation renseignée
SELECT * FROM clients WHERE évaluation IS NULL;

-- Transactions avec montant manquant
SELECT * FROM transactions WHERE montant IS NULL;

-- Compter les valeurs manquantes
SELECT COUNT(*) AS nb_manquants
FROM clients
WHERE évaluation IS NULL;
```

⚠ `AVG()`, `SUM()`, `COUNT()` ignorent les NULL automatiquement. `COUNT(*)` compte toutes les lignes y compris celles avec NULL. `COUNT(colonne)` ne compte que les valeurs non NULL.

12. AS — alias de colonnes et de tables

AS permet de **renommer** une colonne ou une table dans le résultat d'une requête. C'est purement cosmétique — ça ne modifie pas les données.

Alias de colonnes

SQL

```
-- Sans alias - le nom de colonne est la formule entière
SELECT nom, montant * 1.18
FROM transactions;
-- → La colonne s'appelle "montant * 1.18" dans le résultat - illisible

-- Avec alias - nom clair dans le résultat
SELECT nom, montant * 1.18 AS montant_ttc
FROM transactions;
-- → La colonne s'appelle "montant_ttc"

-- Alias avec espaces → guillemets doubles
SELECT nom AS "Nom du client", région AS "Zone géographique"
FROM clients;
```

Alias de tables — raccourcis pour les noms longs

SQL

```
-- Sans alias - verbeux
SELECT clients.nom, clients.région
FROM clients;

-- Avec alias - c remplace clients
SELECT c.nom, c.région
FROM clients c;
-- ou
FROM clients AS c;
```

Les alias de tables deviennent **indispensables** avec les jointures (M08) quand on interroge plusieurs tables simultanément :

SQL

```
-- Sans alias - difficile à lire
SELECT clients.nom, transactions.montant
FROM clients JOIN transactions ON clients.id_client = transactions.id_client;

-- Avec alias - propre et lisible
SELECT c.nom, t.montant
FROM clients c
JOIN transactions t ON c.id_client = t.id_client;
```

💡 Convention courante : utiliser la première lettre du nom de table comme alias — **c** pour **clients**, **t** pour **transactions**, **f** pour **forfaits**.

13. Colonnes calculées dans **SELECT**

On peut créer des **colonnes calculées** directement dans le **SELECT** sans modifier les données sources. C'est l'équivalent SQL des colonnes calculées qu'on ajoutait dans Excel.

Opérations arithmétiques

```
SQL
SELECT
    id_transaction,
    montant,
    montant * 1.18                AS montant_ttc,
    ROUND(montant * 1.18, 0)      AS montant_ttc_arrondi,
    montant / 1000                AS montant_en_milliers
FROM transactions;
```

Fonctions sur les textes

```
SQL
SELECT
    nom,
    UPPER(nom)                AS nom_majuscules,
    LOWER(nom)                AS nom_minuscules,
    LENGTH(nom)               AS longueur_nom,
    région || ' - ' || segment AS description
FROM clients;
-- Note : || est l'opérateur de concaténation en SQLite (= & en Excel)
```

Fonctions sur les dates

SQL

```
SELECT
    nom,
    date_inscription,
    strftime('%Y', date_inscription) AS année_inscription,
    strftime('%m', date_inscription) AS mois_inscription
FROM clients;

-- Note : strftime() est spécifique à SQLite
-- En PostgreSQL : EXTRACT(YEAR FROM date_inscription)
-- En MySQL      : YEAR(date_inscription)
```

CASE WHEN — logique conditionnelle (l'équivalent de SI())

SQL

```
SELECT
    nom,
    montant,
    CASE
        WHEN montant >= 5000 THEN 'Élevé'
        WHEN montant >= 1000 THEN 'Moyen'
        ELSE 'Faible'
    END AS niveau_transaction
FROM transactions;
```



CASE WHEN est l'équivalent SQL du **SI()** imbriqué d'Excel. On le verra en détail dans M09.

14. L'ordre d'exécution SQL — une règle essentielle

En SQL, l'ordre dans lequel on **écrit** les clauses est différent de l'ordre dans lequel elles sont **exécutées**.

Ordre d'écriture (obligatoire)

SQL

```
SELECT  colonnes          -- 1. ce qu'on veut afficher
FROM    table             -- 2. d'où
WHERE   conditions        -- 3. avec quelles conditions
ORDER BY colonne          -- 4. trié comment
LIMIT  n;                -- 5. combien de lignes
```

Ordre d'exécution (interne)

1. FROM → La base de données charge la table
2. WHERE → Elle filtre les lignes
3. SELECT → Elle sélectionne les colonnes
4. ORDER BY → Elle trie
5. LIMIT → Elle coupe

Pourquoi c'est important ?

SQL

```
-- ❌ Cette requête échoue : on ne peut pas utiliser un alias défini
--    dans SELECT à l'intérieur du WHERE
SELECT montant * 1.18 AS montant_ttc
FROM transactions
WHERE montant_ttc > 5000; -- erreur : montant_ttc n'existe pas encore

-- ✅ La bonne façon
SELECT montant * 1.18 AS montant_ttc
FROM transactions
WHERE montant * 1.18 > 5000; -- répéter le calcul dans WHERE
```

💡 Comprendre l'ordre d'exécution explique 80% des erreurs SQL des débutants. Garde ce tableau en tête.

15. Combiner tout — cas pratiques

Voici des requêtes complètes qui combinent tout ce qu'on a vu.

Cas 1 — Explorer la table clients

```
SQL
-- Voir les 5 premières lignes
SELECT * FROM clients LIMIT 5;

-- Quelles régions existent ?
SELECT DISTINCT région FROM clients;

-- Combien de clients au total ?
SELECT COUNT(*) AS nb_clients FROM clients;
```

Cas 2 — Clients Abidjan Postpayé, triés par nom

```
SQL
SELECT nom, région, segment, date_inscription
FROM clients
WHERE région = 'Abidjan'
      AND segment = 'Postpayé'
ORDER BY nom ASC;
```

Cas 3 — Top 10 transactions du mois de mars 2025

```
SQL
SELECT id_transaction, id_client, date, type_produit, montant
FROM transactions
WHERE date BETWEEN '2025-03-01' AND '2025-03-31'
      AND statut = 'Réussi'
ORDER BY montant DESC
LIMIT 10;
```

Cas 4 — Transactions sur des forfaits internet ou voix

SQL

```
SELECT *  
FROM transactions  
WHERE type_produit IN ('Forfait internet', 'Forfait voix')  
      AND statut = 'Réussi'  
ORDER BY date DESC;
```

Cas 5 — Clients dont le nom contient 'Koné'

SQL

```
SELECT *  
FROM clients  
WHERE nom LIKE '%Koné%'  
ORDER BY région;
```

16. Résumé du module

CONCEPT	CE QU'IL FAUT RETENIR
Pourquoi SQL	Volume · temps réel · plusieurs tables · collaboration · reproductibilité
Base de données	Ensemble de tables reliées entre elles
SQL	Langage pour interroger une base — le DA utilise quasi uniquement SELECT
SQLiteOnline	sqliteonline.com — 100% navigateur, zéro install
SELECT	Choisir les colonnes à afficher
FROM	Quelle table interroger
WHERE	Filtrer les lignes selon des conditions
AND / OR	Combiner des conditions
IN	Tester si une valeur est dans une liste
BETWEEN	Tester si une valeur est dans un intervalle
LIKE	Recherche par motif (% = n'importe quoi)
ORDER BY	Trier les résultats (ASC / DESC)
LIMIT	Limiter le nombre de lignes retournées
DISTINCT	Supprimer les doublons
NULL	Absence de valeur — tester avec IS NULL / IS NOT NULL jamais avec =
AS	Alias de colonne ou de table — renommer dans le résultat
Colonnes calculées	Calculs, fonctions texte/date, CASE WHEN directement dans SELECT
Ordre d'exécution	FROM → WHERE → SELECT → ORDER BY → LIMIT

Quiz — Vérifie ta compréhension

Réponds mentalement, puis clique sur **"Voir la réponse"** pour te corriger.

Q1. Tu veux voir toutes les colonnes des 10 premières lignes de la table `clients`. Quelle requête ?

- a) `SELECT 10 FROM clients;`
- b) `SELECT * FROM clients LIMIT 10;`
- c) `SELECT * FROM clients WHERE ligne <= 10;`

👉 Voir la réponse

✅ b) — `SELECT *` sélectionne toutes les colonnes, `FROM clients` indique la table, `LIMIT 10` restreint à 10 lignes. C'est la requête de départ sur toute table inconnue.

Q2. Tu veux les transactions réussies supérieures à 5 000 FCFA, triées du plus grand au plus petit. Quelle requête ?

- a) `SELECT * FROM transactions WHERE montant > 5000 OR statut = 'Réussi' ORDER BY montant DESC;`
- b) `SELECT * FROM transactions WHERE montant > 5000 AND statut = 'Réussi' ORDER BY montant DESC;`
- c) `SELECT * FROM transactions ORDER BY montant DESC WHERE montant > 5000;`

👉 Voir la réponse

✅ b) — `AND` exige que les deux conditions soient vraies simultanément. `OR` retournerait toutes les transactions > 5000 même échouées, et toutes les réussies même < 5000. La requête c) est invalide car `WHERE` doit venir avant `ORDER BY`.

Q3. Quelle est la différence entre `WHERE région = 'Abidjan' OR région = 'Bouaké'` et `WHERE région IN ('Abidjan', 'Bouaké')` ?

- a) `IN` est plus rapide mais moins précis
- b) Aucune différence — les deux retournent le même résultat, `IN` est juste plus lisible
- c) `OR` inclut les valeurs NULL, `IN` non

👉 Voir la réponse

✅ b) — Les deux requêtes retournent exactement le même résultat. `IN` est simplement plus concis et lisible, surtout quand la liste est longue. À partir de 3 valeurs, `IN` est la bonne pratique.

Q4. Tu cherches tous les clients dont le nom contient "Traoré". Quelle clause utiliser ?

- a) `WHERE nom = 'Traoré'`
- b) `WHERE nom IN ('Traoré')`
- c) `WHERE nom LIKE '%Traoré%'`

👉 Voir la réponse

✅ c) — `LIKE '%Traoré%'` trouve toutes les valeurs qui **contiennent** "Traoré" n'importe où dans le nom ("Traoré Brice", "Traoré Fréjus"...). `= 'Traoré'` ne trouverait que les cellules égales exactement à "Traoré".

Q5. Pourquoi cette requête échoue-t-elle ?

SQL

```
SELECT montant * 1.18 AS montant_ttc
FROM transactions
WHERE montant_ttc > 5000;
```

- a) On ne peut pas multiplier dans un `SELECT`
- b) `WHERE` est exécuté avant `SELECT` — l'alias `montant_ttc` n'existe pas encore au moment du filtre
- c) Il manque un `ORDER BY`

👉 Voir la réponse

✓ b) — L'ordre d'exécution SQL est : FROM → WHERE → SELECT. L'alias `montant_ttc` est défini dans `SELECT`, qui s'exécute **après** `WHERE`. Solution : répéter le calcul dans `WHERE` :
`WHERE montant * 1.18 > 5000`.

Q6. Tu veux les 5 clients inscrits le plus récemment. Quelle requête ?

- a) `SELECT * FROM clients LIMIT 5;`
- b) `SELECT * FROM clients ORDER BY date_inscription DESC LIMIT 5;`
- c) `SELECT * FROM clients WHERE date_inscription > '2024-01-01' LIMIT 5;`

👉 Voir la réponse

✓ b) — `ORDER BY date_inscription DESC` trie du plus récent au plus ancien, puis `LIMIT 5` garde les 5 premiers — donc les 5 plus récents. La réponse a) retourne les 5 premières lignes dans l'ordre de stockage, pas les plus récentes.

➡ Module suivant

Tu sais maintenant lire et filtrer des données avec SQL. Dans le **Module 07**, on passe aux agrégations — compter, sommer, grouper — l'équivalent SQL des TCD qu'on a vus en Acte II.

→ [Module 07 — Agrégations et regroupements](#)